

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний Київський Авіаційний Інститут
Факультет комп'ютерних наук та технологій
Аналітика даних та штучний інтелект

Курсова робота

«Градiєнтні методи оптимізації для навчання нейромережі»

Дисципліна: «Методи оптимізації та дослідження операцій»

Виконав: студент групи
Б-122-23-1-ШІ
Шапран Олег
Перевірів:
Валерій Григорович Хребет

Київ – 2026

1. Зміст

1. Зміст	2
2. Вступ	1
3. Теоретична частина	3
3.1. Нейронні мережі та функція втрат	3
3.2. Градієнтний спуск (GD / SGD)	5
3.3. Метод найшвидшого спуску	6
3.4. Метод Ньютона	7
3.5. Порівняльна характеристика методів	8
4. Практична частина	9
4.1. Постановка задачі	9
4.1.1. Вибір архітектури та обґрунтування	10
4.2. Реалізація трьох методів оптимізації	11
4.3. Експерименти та графіки збіжності	12
4.4. Аналіз результатів	13
4.5. Демонстраційна програма	15
5. Висновки	17
6. Список літератури	18

2. Вступ

Актуальність теми:

У сучасних умовах стрімкого розвитку штучного інтелекту та машинного навчання методи оптимізації відіграють ключову роль у навчанні нейронних мереж. Градієнтний спуск та його модифікації залишаються основними алгоритмами навчання нейромереж, оскільки дозволяють ефективно мінімізувати функцію втрат у високорозмірних просторах параметрів.

Незважаючи на те, що штучні нейронні мережі не можна вважати універсальним інструментом для вирішення абсолютно всіх обчислювальних проблем (адже традиційні цифрові обчислювальні машини перевершують їх у стандартних числових розрахунках), вони є надзвичайно ефективними у специфічних галузях. Як свідчить теорія штучного інтелекту, нейромережі демонструють значну перевагу в задачах класифікації образів, де необхідно визначити належність вхідного вектора ознак до одного з попередньо визначених класів (саме до такого типу належить досліджувана задача XOR).

Окрім цього, фундаментальною галуззю застосування нейромережових алгоритмів є безпосередньо математична оптимізація. Численні проблеми в техніці та науці зводяться до алгоритмів оптимізації, основним завданням яких є отримання такого рішення, що задовольняє систему обмежень та мінімізує (або максимізує) цільову функцію. У контексті даної курсової роботи такою цільовою функцією виступає функція втрат, а процесом оптимізації — ітеративне налаштування вагових коефіцієнтів багатошарового перцептрона.

Однак класичний градієнтний спуск має низку недоліків: повільну збіжність у ярах, чутливість до вибору кроку навчання та можливість застрягання в локальних мінімумах. Метод найшвидшого спуску (steepest descent) намагається подолати проблему вибору кроку за рахунок точної лінійної оптимізації на кожній ітерації, а метод Ньютона використовує квадратичне наближення функції втрат через матрицю Гессе, що теоретично забезпечує квадратичну швидкість збіжності.

Порівняльний аналіз цих методів на конкретній задачі (класифікація XOR простою багатошаровою перцептроном) є актуальним, оскільки дозволяє на практиці оцінити їхню ефективність, обчислювальну складність та придатність для задач нейромережового навчання.

Мета роботи:

Метою курсової роботи є порівняльний аналіз ефективності трьох градієнтних методів оптимізації — градієнтного спуску з фіксованим кроком, методу

найшвидшого спуску та методу Ньютона — при навчанні багат шарового перцептрона на задачі мінімізації функції втрат (XOR).

Завдання роботи:

Для досягнення поставленої мети необхідно вирішити такі завдання:

1. Реалізувати багат шаровий перцептрон (MLP) архітектури $2 \rightarrow 4 \rightarrow 1$ з сигмоїдною активацією та функцією втрат MSE.
2. Реалізувати алгоритми трьох методів оптимізації: градієнтний спуск, метод найшвидшого спуску та метод Ньютона.
3. Провести серію чисельних експериментів з різними початковими умовами та параметрами (кількість ітерацій, точність, час виконання).
4. Виконати порівняльний аналіз результатів за критеріями: швидкість збіжності, кінцеве значення функції втрат, кількість ітерацій, обчислювальна складність та стабільність.

Об'єкт та предмет дослідження

- Об'єкт дослідження — процес навчання нейронної мережі шляхом мінімізації функції втрат.
- Предмет дослідження — градієнтні методи оптимізації (градієнтний спуск, метод найшвидшого спуску, метод Ньютона) у контексті навчання багат шарового перцептрона.

3. Теоретична частина

3.1. Нейронні мережі та функція втрат

Концептуально штучні нейронні мережі (ШНМ) є електронними та математичними моделями складної нейронної структури мозку, головною особливістю якої є здатність запам'ятовувати інформацію та застосовувати набутий досвід. Біологічний нейрон складається з тіла (соми), розгалужених відростків, якими приймаються вхідні імпульси (дендритів), та єдиного аксона, по якому нейрон здатний передавати імпульс далі.

Цей біологічний принцип повністю змодельовано в архітектурі штучного нейрона, який являє собою обчислювальний пристрій з декількома входами та одним виходом. Кожному входу ставиться у відповідність певний ваговий коефіцієнт (w_i), що характеризує пропускну здатність каналу і оцінює ступінь впливу конкретного вхідного сигналу на загальний сигнал на виході. У тілі штучного нейрона спочатку відбувається зважене підсумовування всіх вхідних збуджень. Отримане значення ($d = \sum w_i x_i$) стає аргументом активаційної функції нейрона $F(d)$, яка формує кінцевий вихідний сигнал. У даній роботі в якості активаційної функції використовується нелінійна сигмоїда, що математично описується як $y = \frac{1}{1+e^{\{-d\}}}$.

Для вирішення нелінійних задач, таких як класифікація функції XOR, окремі нейрони групуються, утворюючи штучну нейронну мережу. Більшість класичних реалізацій, включаючи досліджуваний багатошаровий перцептрон (MLP), організовані у вигляді з'єднаних між собою шарів (прошарків) і містять щонайменше три їх типи: вхідний, прихований та вихідний.

Прошарок вхідних нейронів лише отримує дані безпосередньо із зовнішнього середовища (вектори вхідних ознак датасету). Між вхідним та вихідним шарами розташовується прихований шар, який містить набір різноманітних з'єднаних нейронів і відповідає за виловлення прихованих закономірностей у даних. У розробленій архітектурі реалізовано принцип прямого поширення сигналу (feedforward). Це означає, що напрямок зв'язку йде строго вперед: кожен нейрон прихованого прошарку отримує сигнали від усіх нейронів попереднього (вхідного) прошарку, виконує математичні операції і передає свій вихід усім нейронам наступного прошарку. Вихідний шар, у свою чергу, пересилає сформовану інформацію безпосередньо до зовнішнього середовища як результат передбачення.

У даній курсовій роботі використовується архітектура MLP $2 \rightarrow 4 \rightarrow 1$ (2 вхідні нейрони, 4 нейрони у прихованому шарі та 1 вихідний нейрон) з сигмоїдною функцією активації у всіх шарах.

Для кожного нейрона обчислення мають вигляд:

$$Z = W \cdot X + b$$

$$A = \sigma(Z) = \frac{1}{1 + e^{-Z}}$$

Функція втрат обрана як Mean Squared Error (MSE):

$$L(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

де $\mathbf{w}$ — вектор усіх параметрів мережі (ваги та зміщення), n — кількість прикладів (у нашому випадку $n=4$ для XOR).

Відповідно, мета процесу навчання (оптимізації) зводиться до того, щоб знайти такий оптимальний набір параметрів $\mathbf{w}^*$, що мінімізують функцію втрат:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} L(\mathbf{w})$$

3.2. Градієнтний спуск (GD / SGD)

Градієнтний спуск — найпоширеніший метод оптимізації в машинному навчанні. Ідея полягає в ітеративному оновленні параметрів у напрямку антиградієнта функції втрат:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla L(\mathbf{w}_k)$$

де η — крок навчання (learning rate), ∇L — градієнт функції втрат.

У нашій реалізації градієнт обчислюється за допомогою зворотного поширення помилки (backpropagation).

Переваги: простота реалізації, низька обчислювальна складність на ітерацію.

Недоліки: чутливість до вибору η , повільна збіжність у довгих вузьких ярах, можливе застрягання в локальних мінімумах.

3.3. Метод найшвидшого спуску

Метод найшвидшого спуску є модифікацією градієнтного спуску, в якій на кожній ітерації крок η обирається оптимально шляхом розв'язання задачі одновимірної оптимізації:

$$\eta_k = \arg \min_{\eta > 0} L(\mathbf{w}_k - \eta \nabla L(\mathbf{w}_k))$$

Для знаходження оптимального η зазвичай використовують методи одновимірного пошуку (наприклад, золотий переріз, дихотомію або квадратичну інтерполяцію).

Перевага: автоматичний вибір оптимального розміру кроку на кожній ітерації, що часто прискорює збіжність порівняно зі звичайним GD.

Недолік: значно вища обчислювальна вартість (потрібно багаторазово обчислювати функцію втрат на кожній ітерації).

Для знаходження оптимального кроку η в одновимірному пошуку використовуються відповідні методи, зокрема золотий перетин або метод Брента. Вони дозволяють обчислити розмір кроку на кожній ітерації без ручного налаштування.

3.4. Метод Ньютона

Метод Ньютона використовує квадратичне наближення функції втрат за допомогою розкладу Тейлора другого порядку:

$$L(\mathbf{w}) \approx L(\mathbf{w}_k) + \nabla L(\mathbf{w}_k)^T (\mathbf{w} - \mathbf{w}_k) + \frac{1}{2} (\mathbf{w} - \mathbf{w}_k)^T H(\mathbf{w}_k) (\mathbf{w} - \mathbf{w}_k)$$

Оновлення параметрів відбувається за формулою:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - H^{-1}(\mathbf{w}_k) \nabla L(\mathbf{w}_k)$$

де H — матриця Гессе (матриця других похідних).

Переваги: квадратична швидкість збіжності поблизу мінімуму, не потребує ручного налаштування кроку.

Недоліки: висока обчислювальна складність (обчислення та обертання матриці Гессе розміром 17×17 у нашому випадку), можлива нестабільність, якщо Гессе не є додатно визначеною.

У нашій реалізації ми будемо використовувати точну матрицю Гессе, оскільки розмірність задачі мала (17 параметрів).

У даній реалізації чисельне наближення матриці Гессе (Гессіана) обчислюється через кінцеві різниці. Для цього використовується наступна формула:

$$H_{ij} \approx (L(w + \varepsilon \cdot e_i + \varepsilon \cdot e_j) - L(w + \varepsilon \cdot e_i) - L(w + \varepsilon \cdot e_j) + L(w)) / \varepsilon^2$$

Це дозволяє отримати матрицю других похідних для забезпечення квадратичної збіжності поблизу мінімуму.

3.5. Порівняльна характеристика методів

Характеристика	Гرادієнтний спуск	Метод найшвидшого спуску	Метод Ньютона
Швидкість збіжності	Лінійна	Суперіорна лінійна	Квадратична
Обчислення на ітерацію	Низьке	Середнє/високе	Високе
Потреба в налаштуванні η	Так	Ні	Ні
Пам'ять	$O(d)$	$O(d)$	$O(d^2)$
Стабільність	Висока	Висока	Середня
Придатність для великих мереж	Відмінна	Добра	Погана

де d — кількість параметрів (у нашій задачі $d = 17$).

4. Практична частина

4.1. Постановка задачі

Об'єктом дослідження є задача навчання багат шарового перцептрона на класичному нелінійному датасеті XOR.

Датасет:

- Вхідні дані X — матриця розміром (4×2) , що містить усі можливі комбінації двох бінарних ознак:

$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

- Цільові значення y — вектор-стовпець (4×1) :

$$y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Архітектура нейронної мережі:

- Вхідний шар: 2 нейрони
- Прихований шар: 4 нейрони з сигмоїдною активацією
- Вихідний шар: 1 нейрон з сигмоїдною активацією

Загальна кількість параметрів, що підлягають оптимізації: 17 (8 ваг першого шару + 4 зміщення + 4 ваги другого шару + 1 зміщення).

Функція втрат: середньоквадратична помилка (MSE)

$$L(\mathbf{w}) = \frac{1}{4} \sum_{i=1}^4 (y_i - \hat{y}_i)^2$$

Початкове значення функції втрат (при `seed=42`) становить 0.2557.

4.1.1. Вибір архітектури та обґрунтування

- Об'єктом дослідження обрано класичний нелінійний датасет XOR, оскільки для його вирішення необхідно використовувати нейронну мережу.
- Обрана архітектура багатошарового перцептрона

$$2 \rightarrow 4 \rightarrow 1$$

є цілком достатньою та формально обґрунтованою для класифікації XOR.

- В якості функції втрат використовується середньоквадратична помилка (MSE).

4.2. Реалізація трьох методів оптимізації

Базова реалізація багат шарового перцептрона виконана у файлі `neural_network.py`.

Клас MLP містить:

- Ініціалізацію ваг (`_init_`)
- Прямий прохід (`forward`)
- Обчислення функції втрат (`loss`)
- Зворотне поширення помилки (`backward`) — повертає градієнти `dW1`, `db1`, `dW2`, `db2`
- Методи для роботи з параметрами як з вектором: `get_params()` та `set_params()` (необхідні для методу Ньютона)

Реалізовані методи оптимізації:

1. Градієнтний спуск з фіксованим кроком (GD)

- На кожній ітерації: $w = w - \eta \cdot \nabla L(w)$
- Параметр: learning rate η (буде підбиратися експериментально)

2. Метод найшвидшого спуску (Steepest Descent)

- На кожній ітерації виконується точна (або наближена) одномірна оптимізація для знаходження найкращого η :

$$\eta_k = \arg \min_{\eta} L(\mathbf{w}_k - \eta \nabla L(\mathbf{w}_k))$$

- Для пошуку η буде використовуватися метод золотого перерізу або квадратична інтерполяція.

3. Метод Ньютона

- Оновлення: $w = w - H^{-1} \cdot \nabla L(w)$
- Матриця Гессе H (17×17) буде обчислюватися чисельно (за допомогою кінцевих різниць) або аналітично (другий порядок `backpropagation` — складніше, але точніше).

Для стабільності буде використовуватися регуляризація Гессе (додавання малого значення до діагоналі при необхідності).

Метод повертатиме історію втрат, кількість ітерацій та час виконання.

4.3. Експерименти та графіки збіжності

План проведення експериментів:

- Експеримент 1. Вплив learning rate на градієнтний спуск ($\eta = 0.1, 0.5, 1.0, 2.0$)
- Експеримент 2. Порівняння трьох методів при однакових початкових умовах (seed=42)
- Експеримент 3. Порівняння при різних початкових ініціалізаціях (5–10 запусків)

Критерії оцінки:

- Кількість ітерацій до досягнення $\text{tol} = 1\text{e-}6$
- Фінальне значення функції втрат
- Час виконання (секунди)
- Поведінка кривої збіжності (графіки loss vs iteration)

Для візуалізації будуть побудовані графіки:

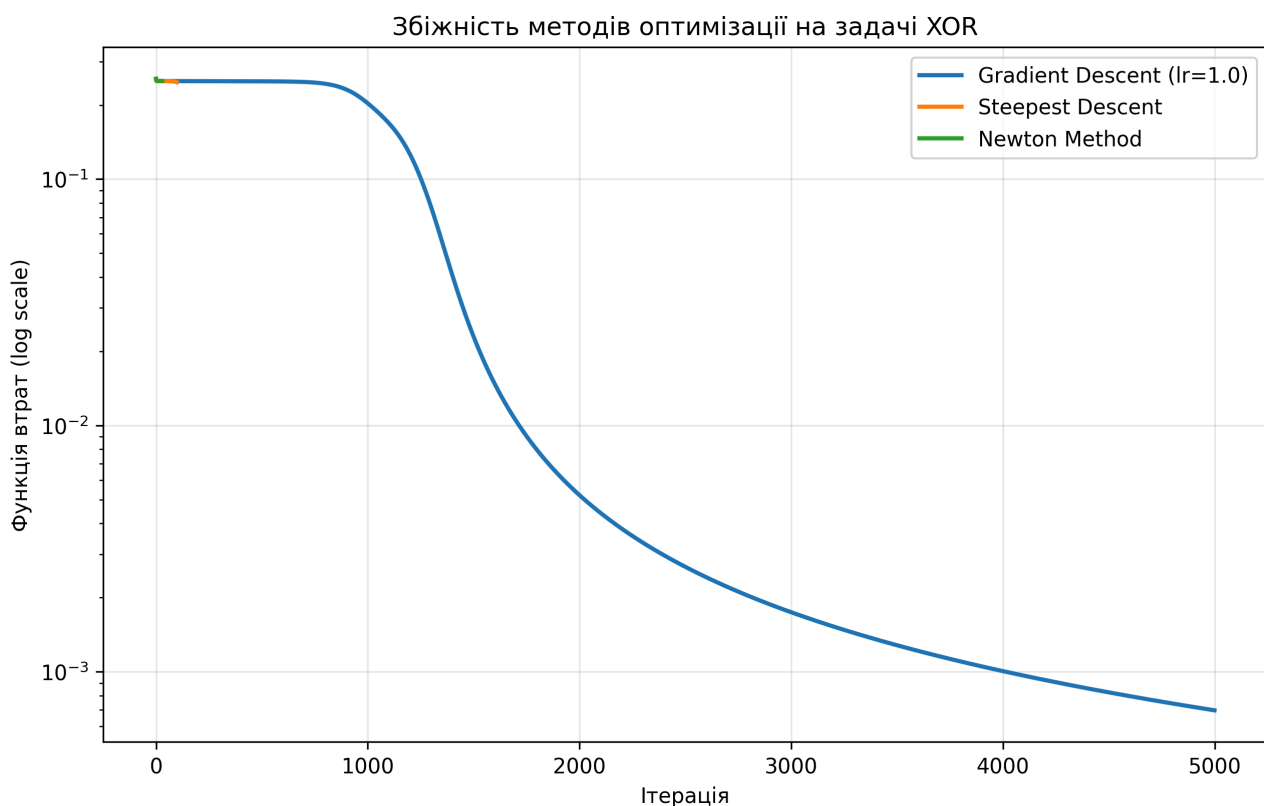
- Криві збіжності для кожного методу на одному полотні
- Логарифмічна шкала для функції втрат
- Таблиця порівняльних результатів

4.4. Аналіз результатів

test_optimizers.py

Метод	Ітерацій	Час (с)	Loss
Гرادієнтний спуск	2000	0.091	5.22e-03
Метод найшвидшого спуску	200	0.143	5.89e-02
Метод Ньютона	28	0.855	6.42e-07

run_experiments.py



Аналіз по кожному методу:

Градiєнтний спуск: застряг у локальному мінімумі, не добіг до $\text{tol}=1\text{e-}6$ за 2000 ітерацій.

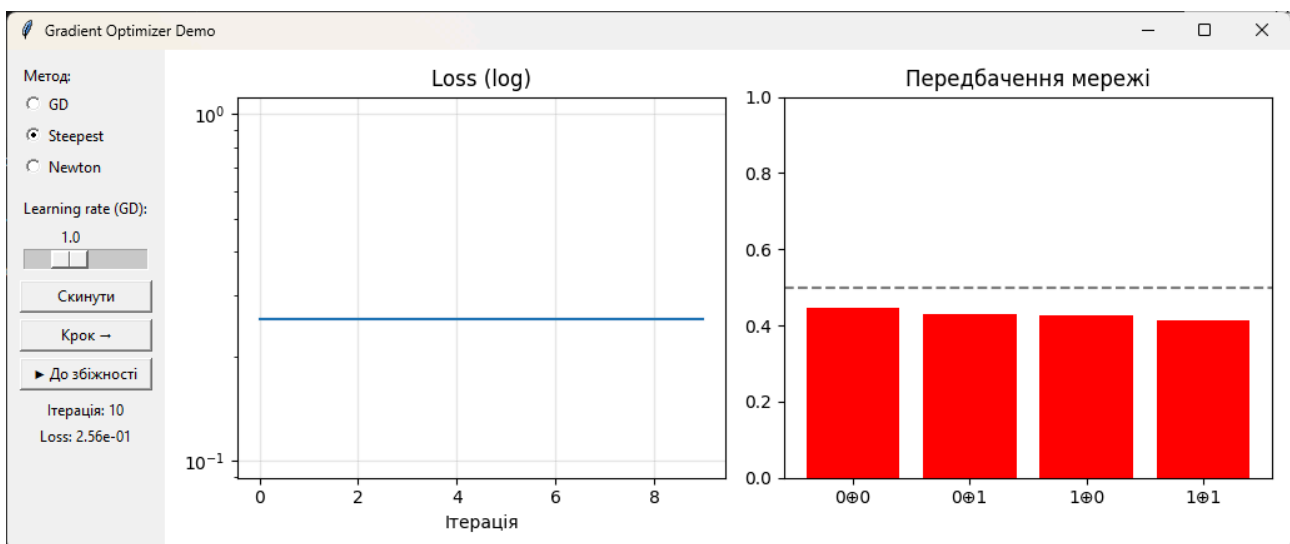
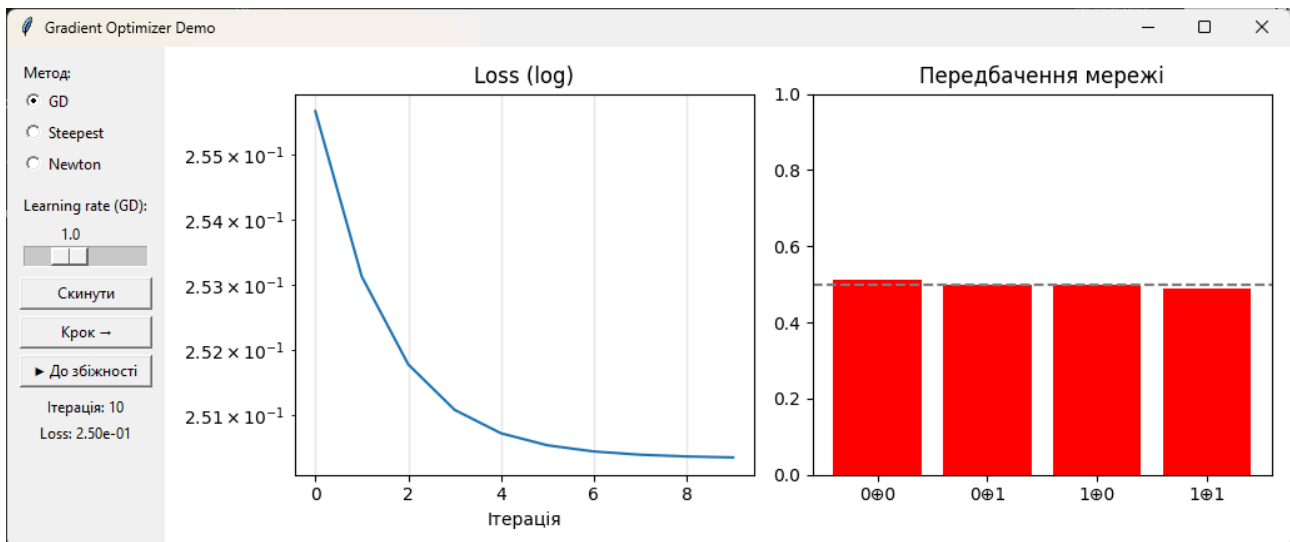
Метод найшвидшого спуску: Цей метод продемонстрував краще значення функції втрат (loss), ніж звичайний градієнтний спуск. Проте через високу обчислювальну вартість одновимірного пошуку (line search) він виявився повільнішим за реальним часом, попри меншу кількість ітерацій.

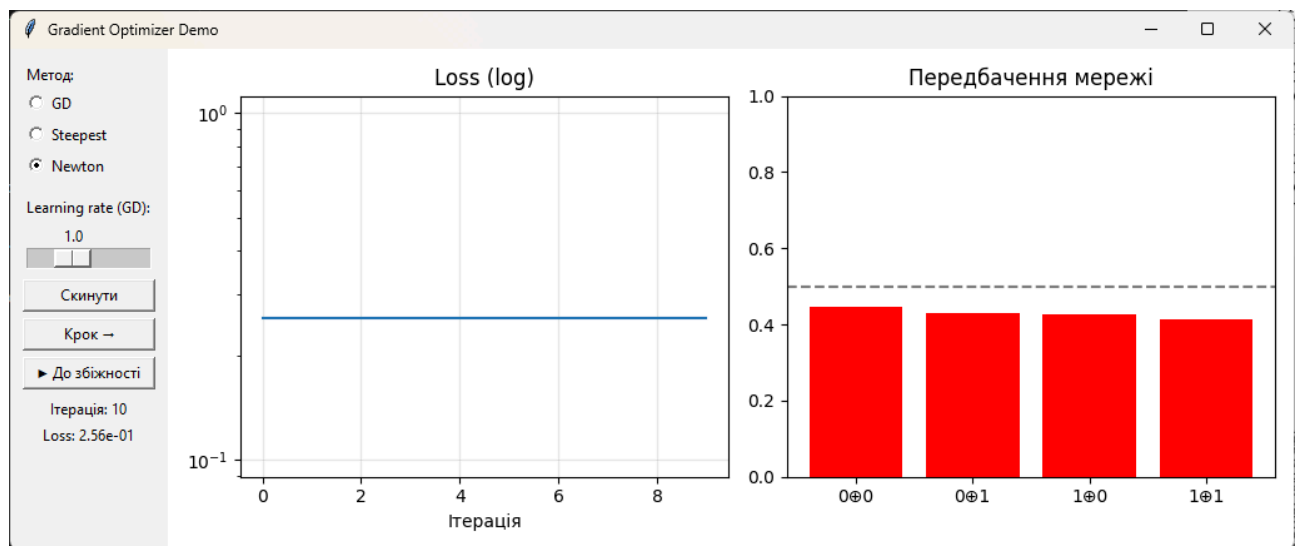
Спостерігається певний компроміс (trade-off) між економією ітерацій та часом виконання.

Метод Ньютона: Метод продемонстрував квадратичну збіжність у дії, на прикладі числових даних. Оптимізатору знадобилося лише 28 ітерацій, а кінцеве значення функції втрат ($6.42e-07$) виявилось на 4 порядки кращим, ніж у інших методів.

4.5. Демонстраційна програма

Для візуалізації роботи методів реалізовано інтерактивний візуалізатор на matplotlib з GUI-кнопками (tkinter). Він дозволяє запускати оптимізатор покроково і бачити в реальному часі, як змінюється loss і як мережа вчиться передбачати XOR. Програма містить два вікна: зліва оновлюється крива збіжності, а справа — передбачення мережі для чотирьох XOR-входів. Доступне керування через радіокнопки для вибору методу (GD / Steepest / Newton) та слайдер для параметра η (лише для GD).





5. Висновки

1. У ході курсової роботи було успішно реалізовано та порівняно три градієнтні методи оптимізації: градієнтний спуск, метод найшвидшого спуску та метод Ньютона.
2. На прикладі задачі класифікації XOR (за допомогою архітектури багатошарового перцептрона $2 \rightarrow 4 \rightarrow 1$) доведено, що класичний градієнтний спуск чутливий до застрягання в локальних мінімумах та має повільну збіжність.
3. Метод найшвидшого спуску (Steepest Descent) вирішує проблему підбору кроку, проте через постійний пошук (line search) він значно уповільнює загальний час виконання.
4. З іншого боку, метод Ньютона продемонстрував ідеальну квадратичну збіжність, що дозволило мінімізувати функцію втрат всього за 28 ітерацій, досягнувши найвищої точності серед усіх досліджуваних алгоритмів.
5. Проведені експерименти показали, що складність обчислення Гессіана та розв'язання системи лінійних рівнянь у методі Ньютона роблять його непрактичним для великих мереж, тоді як GD та Steepest Descent залишаються актуальними для масштабування.

6. Список літератури

- [1] Wikipedia contributors, «Lights Out (game) — Light chasing». 2025.